

weak_ptr — Խորը ուղեցույց

C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::weak_ptr`-ը՝ ինչ է, ինչու է պետք, circular reference-ի լուծում, `lock()`, `expired()`, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է `weak_ptr`-ը

`std::weak_ptr`-ը «թուլացրած» (non-owning) smart pointer է, որ **ցույց է տալիս `shared_ptr`-ի կառավարած resource-ին, բայց ՉԻ ավելացնում reference count-ը**:

```
#include <memory>

auto sp = std::make_shared<int>(10);    // strong count = 1
std::weak_ptr<int> wp = sp;              // strong count ԴԵՆ 1 (weak չի ավելացնում)

cout << sp.use_count();    // 1 (weak_ptr չի հաշվվում)
```

`weak_ptr`-ը «դիտորդ» է՝ գիտի resource-ի մասին, բայց ownership չունի, չի երկարացնում կյանքը:

2. Ինչու է պետք — Circular Reference-ի լուծում

`shared_ptr`-ի ամենամեծ problem-ը՝ circular reference (leak): `weak_ptr`-ը լուծում է այն:

Խնդիր (shared_ptr-ով)

```
struct Node {
    std::shared_ptr<Node> next;
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b count = 2
b->next = a;    // a count = 2
// scope-ի վերջ -> count 1 (իրար պահում) -> երբեք 0 -> LEAK
```

Լուծումը (weak_ptr-ով)

```
struct Node {
    std::weak_ptr<Node> next;    // weak -> count չի ավելացնում
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b strong count ԴՆՌ 1
b->next = a;    // a strong count ԴՆՌ 1
// scope-ի վերջ -> count 0 -> DELETE -> ՉԿԱ LEAK
```

Parent-child կապերում՝ parent -> child `shared_ptr`, child -> parent `weak_ptr` (կոտրում է cycle-ը):

3. lock() — անվտանգ մուտք resource-ին

`weak_ptr`-ից ուղիղ չես կարող dereference անել (resource-ը կարող է ջնջված լինել): Պետք է `lock()` -> վերադարձնում `shared_ptr` (եթե resource-ը կա):

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;

if (auto locked = wp.lock()) {    // shared_ptr վերադարձնում (count++ ժամանակավոր)
    cout << *locked;              // անվտանգ -- resource կա
} else {
    cout << "resource ջնջված է";
}
```

`lock()` -ը. եթե resource-ը կա -> valid shared_ptr; եթե ջնջված -> nullptr shared_ptr:

4. expired() — ստուգում resource-ը կա՞նք

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;

cout << wp.expired();    // false (կա)
sp.reset();              // strong count 0 -> resource DELETE
cout << wp.expired();    // true (ջնջված է)
```

`expired()` -> `true` եթե resource-ը ջնջված է (strong count == 0):

5. Կարևոր մեթոդներ

```
std::weak_ptr<int> wp = sp;

wp.lock();              // shared_ptr վերադարձնում (կամ nullptr)
wp.expired();           // resource ջնջված է?
wp.use_count();         // քանի STRONG owner կա
wp.reset();             // weak-ը դատարկացնում
```

weak_ptr-ից **ՉԵՍ կարող** ուղիղ `*wp` կամ `wp->` -- պետք է `lock()` :

6. Իրական օրինակ — Observer / Cache

```
class Cache {
    std::unordered_map<int, std::weak_ptr<Data>> cache;
public:
    std::shared_ptr<Data> get(int key) {
        if (auto sp = cache[key].lock())    // դեռ կա՞
            return sp;
        auto sp = loadFromDisk(key);      // վերանորոգել
        cache[key] = sp;                   // weak -> չի երկարացնում կյանքը
        return sp;
    }
};
```

Cache-ը `weak_ptr` է պահում -> չի խանգարում resource-ի ազատմանը, բայց կարող է «վերագտնել» եթե դեռ կա:

7. Հարցազրույցի հարցեր և պատասխաններ

`weak_ptr` ի՞նչ է:

Non-owning smart pointer, որ ցույց է տալիս `shared_ptr`-ի resource-ին բայց չի ավելացնում `strong count`-ը; չի երկարացնում կյանքը:

Ինչո՞ւ է պետք:

Circular reference-ի լուծում (leak); observer/cache patterns:

Circular reference ինչպես է լուծում:

Մեկ ուղղությամբ `weak_ptr` (`parent->child shared, child->parent weak`) -> cycle կոտրվում -> `count 0 -> delete`:

`lock()` ինչ է անում:

Վերադարձնում `shared_ptr (count++)`, եթե `resource`-ը կա: Հակառակ դեպքում `nullptr`.
Անվտանգ մուտքի ձև:

Ինչո՞ւ ուղիղ չես `dereference` անում `weak_ptr`:

Resource-ը կարող է ջնջված լինել: պետք է `lock()` -> ստուգել կա՞ թե ոչ:

`expired()` ի՞նչ է:

`true` եթե `resource` ջնջված է (`strong count 0`):

`weak_ptr` ավելացնում է `use_count`-ը:

Ոչ -- միայն `strong (shared_ptr) count`. `weak count` առանձին է:

Ամփոփում (Cheat Sheet)

`weak_ptr = NON-OWNING` (չի ավելացնում `strong count`, չի երկարացնում կյանք)

<code>wp = shared_ptr</code>	-> weak reference (<code>strong count</code> ԴԵՌԻ ԼՈՒՂԵՐ)
<code>wp.lock()</code>	-> <code>shared_ptr</code> (կամ <code>nullptr</code> եթե ջնջված) -- ԱՆՎՏԱՆԳ մուտք
<code>wp.expired()</code>	-> resource ջնջված է? (<code>strong count 0</code>)
<code>wp.use_count()</code>	-> քանի <code>STRONG owner</code>
<code>*wp / wp-></code>	-> ՉԿԱ (պետք է <code>lock()</code>)

ՀԻՍՏԱԿԱՆ ԿԻՐԱՌ: `circular reference` լուծում (`parent->child shared`, `child->parent weak`)
`observer / cache patterns`